

day 3

JavaScript Core Concepts: Types, Strings, RegEx, Math & Arrays

1. Type Conversion & Coercion (تحويل الأنواع)

في الجافا سكريبت، نقدر نحول البيانات من نص لرقم أو العكس بكذا طريقة، وكل طريقة لها سلوك مختلف تماماً:

التحويل إلى رقم (To Number)

* أسرع طريقة لتحويل النص لرقم (Unary Plus): '+' علامة الـ

```
```javascript
```

```
var num = "ali";
```

```
console.log(typeof num); // string
```

```
console.log(+num); // NaN (لأن الكلمة لا يمكن تمثيلها كرقم)
```

```
console.log(typeof +num); // number (في جافا سكريبت هو NaN تنبيه: نوع ⚠️ number!)
```

• كائن `Number()` المدمج: يشغل زي علامة الـ + بالطبط؛ لو النص فيه أي حرف مش رقم بيرجع `NaN`.

```
var str1 = "25.6";
```

```
var str2 = "25.3%";
```

```
console.log(+str1); // 25.6 (Number)
```

```
console.log(+str2); // NaN (بسبب علامة %)
```

```
console.log(Number(str1)); // 25.6
```

```
console.log(Number(str2)); // NaN
```

• الدوال الذكية (`parseInt` & `parseFloat`): بتستخرج الأرقام من النصوص حتى لو فيها رموز (بشرط النص يبدأ برقم).

- `parseInt()`: يتحول النص لرقم صحيح وتطير الكسر تماماً وتطير الرموز.
- `parseFloat()`: يتحول النص لرقم عشري (بالكسر بتاعه) وتطير الرموز بس.

JavaScript

```
parseInt //
```

```
console.log(parseInt(str1)); // 25 (شال الـ 6.)
```

```
console.log(parseInt(str2)); // 25 (شال الـ 3%)
```

```
parseFloat //
```

```
console.log(parseFloat(str1)); // 25.6
```

```
console.log(parseFloat(str2)); // 25.3 (طير الـ % وساب الكسر)
```

ب) دالة التنفيذ الديناميكي `eval()`

- **eval(string)** : JavaScript. يتأخذ نص عبارة عن كود جافا سكريبت أو معادلة رياضية وتنفذها فوراً وترجع الناتج :

```
var str3 = "2+3*20-8";
console.log(eval(str3)); // 54 (54 = 8-60+2 ثم 60=20*3 أولاً)
```

## ج) اختبارات الفهم وتداخل الدوال (Tricky Examples)

JavaScript

```
var num = 5;
console.log(String(num)); // "5"
console.log(typeof String(num)); // string

var numStr = "5";
console.log(typeof Number(numStr)); // number
console.log(Number(typeof numStr)); // Number("string") -> NaN
console.log(Number("string")); // NaN
```

## 2. Strings Deep Dive (النصوص بالتفصيل)

### أ) طرق إنشاء النص والفرق بين الـ Primitive والـ Object

1. **String Primitive:** string الطريقة العادية والمفضلة، نوعها يبقى .
2. **String Object:** object نوعه يبقى ، new String() إنشاؤه باستخدام .

JavaScript

```
// Primitive
var str1 = "ali";
console.log(str1[1]); // "l"
console.log(str1.length); // 3
console.log(typeof str1); // string

// Object
var str3 = new String("ahmed");
console.log(str3[1]); // "h"
console.log(str3.length); // 5
console.log(typeof str3); // object
```

### ب) مفهوم الفهارس (Indexes) في النصوص

العد يبدأ من 0 من البداية، أو بالسالب يبدأ من -1 من النهاية:

a	h	m	e	d	Length = 5
4	3	2	1	0	:المؤشر+
1-	2-	3-	4-	5-	:المؤشر-

## (ج) أهم ميثودز النصوص (String Methods)

### JavaScript

```
var str = "ahmeddd"; // Length: 7, Index: 0 to 6 (Negative: -7 to -1)

// 1. charAt(index): بتجيب الحرف اللي في المكان ده
console.log(str.charAt(4)); // "d"
console.log(str[4]); // "d"

// 2. indexOf(char): بتجيب أول مكان ظهر فيه الحرف (البحث من البداية)
console.log(str.indexOf("d")); // 4

// 3. lastIndexOf(char): بتجيب آخر مكان ظهر فيه الحرف (البحث من النهاية)
console.log(str.lastIndexOf("d")); // 6

// 4. toUpperCase() / toLowerCase(): تحويل حالة الأحرف
console.log(str.toUpperCase()); // "AHMEDDD"

// 5. trim(): إزالة المسافات من البداية والنهاية فقط
var strWithSpaces = " ahmed ";
console.log(strWithSpaces.trim()); // "ahmed"

// 6. substring(from, to): (⚠️ لا تشمل مؤشر النهاية) قص جزء من النص
console.log(str.substring(1, 4)); // "hme" (من اندكس 1 لـ 3)

// 7. slice(start, end): بس بتدعم الفهارس السالبة من substring بتشتغل زي
// النهاية
console.log(str.slice(1, 4)); // "hme"
console.log(str.slice(-4, -1)); // "edd" (من اندكس -4 لـ -2)
```

## (د) دالة التقطيع split() وتطبيقاتها الفصيلية

- **split(delimiter, limit):** اختياري لتحديد عدد limit بناءً على الفاصل الممرر ليها، والـ Array بتحول النص لـ العناصر المستخرجة.

### JavaScript

```
var sentence = "Hello world how are you";
console.log(sentence.split(" ")); // ['Hello', 'world', 'how', 'are',
```

```
'you']
console.log(sentence.split(" ", 3)); // ['Hello', 'world', 'how'] (أول 3 عناصر)
(بس)

var sentence2 = "Hello*world*how*are*you";
console.log(sentence2.split("*")); // ['Hello', 'world', 'how', 'are', 'you']
```

## 🔥 تطبيق عملي متقدم (استخراج قيم الـ Query String من الروابط):

لو عندنا نص جاي من الرابط بالشكل ده وعابزين نطلع القيم اللي بعد علامة يساوي = علطول:

### JavaScript

```
var myString = "name=ahmed&age=20&gender=male&hobby=football";
var splittedString = myString.split("&");
// splittedString هتبقى: ['name=ahmed', 'age=20', 'gender=male', 'hobby=football']

for(var i = 0 ; i < splittedString.length ; i++){
 // تاني عند الـ "=" واخذ العنصر الثاني اللي هو الاندكس split كل عنصر هعمله
 [1]
 console.log(splittedString[i].split("=")[1]);
}
// المخرجات هتبقى:
// "ahmed"
// "20"
// "male"
// "football"
```

### 🔗 تذكير سريع بالوصول للبيانات

- الوصول لعنصر في المصفوفة: `arr[3]`
- الوصول لقيمة داخل `Object`: `object.a` أو `object['a']`

## 3. Regular Expressions (التعابير المنتظمة) Regex

تستخدم للتحقق من صياغة المدخلات (Validation) مثل الإيميلات أو صيغ النصوص:

- `test(text)`: دالة بتختبر النص لو متطابق مع النمط ترجع `true` وإلا `false`.

### JavaScript

```
var regexEmail = new RegExp("[^@ \\t\\r\\n]+@[^@ \\t\\r\\n]+\\.([^@ \\t\\r\\n]+)"); // نمط الإيميل

console.log(regexEmail.test("hello@")); // false (إيميل غير كامل)
console.log(regexEmail.test("geon@ihateregex.io")); // true (إيميل صيغته صحيحة)
```

## استخدامها مع حلقات التكرار (do...while)

كود يجبر المستخدم على إدخال إيميل صحيح، ولن يخرج من اللوب إلا إذا تحقق الشرط:

JavaScript

```
do {
 var res = prompt("enter your email");
} while(regexEmail.test(res) === false); // false طول ما النتيجة
الطلب

console.log(res); // هيطبع الإيميل الصح بعد الخروج من اللوب
```

## 4. Math Object (العمليات الحسابية المتقدمة)

كائن Math هو **Static Class** (يعني بنادي عليه مباشرة باسمه بدون استخدام new).

JavaScript

```
console.log(Math.pow(2, 3)); // 8 (أس 2 3)
console.log(Math.sqrt(16)); // 4 (الجذر التربيعي لـ 16)
console.log(Math.min(20, 5, 60, 30, 70, 10)); // 5 (أقل قيمة)
console.log(Math.max(20, 5, 60, 30, 70, 10)); // 70 (أعلى قيمة)
console.log(Math.random()); // رقم عشوائي بين 0 (مغلق) و 1 (مفتوح)
```

## هام جداً: مقارنة دوال التقريب (Floor vs Ceil vs Trunc)

الفرق بيظهر بوضوح جداً مع الأرقام السالبة:

الدالة	الوظيفة	مثال موجب 5.7	مثال سالب -5.7
Math.floor()	التقريب لـ الأقل دائماً (لأسفل/الأرض)	5	6- (لأن 6 أصغر من -5.7)
Math.ceil()	التقريب لـ الأعلى دائماً (لأعلى/السقف)	6	5- (لأن 5 أكبر من -5.7)
Math.trunc()	حذف الكسر تماماً بدون تقريب (Truncate)	5	-5

```
console.log(Math.floor(5.7)); // 5
console.log(Math.ceil(5.7)); // 6
console.log(Math.trunc(5.7)); // 5

console.log(Math.floor(-5.7)); // -6
console.log(Math.trunc(-5.7)); // -5
```

## 5. Arrays Deep Dive (المصفوفات بالتفصيل)

### أ) طرق إنشاء المصفوفة ونوعها

سواء أنشأتها بالطريقة العادية أو بـ `new Array()` ، الـ `typeof` دائماً `object`.

```
var fruits = ["apple", "banana"];
console.log(typeof fruits); // object

var fruitsObj = new Array("mango", "watermelon");
console.log(typeof fruitsObj); // object
```

### ب) ميثودز المصفوفات (Array Methods)

تنقسم الميثودز لنوعين (نوع يحافظ على المصفوفة الأصلية ونوع يعدلها):

#### 1. ميثودز لا تغير المصفوفة الأصلية (Immutable - Returns New Value)

- `concat()` : دمج المصفوفات.
- `join(separator)` : تحويل المصفوفة لنص وبينهما الفاصل الممرر.
- `toString()` : تحويل المصفوفة لنص والفاصل الافتراضي دائماً الفاصلة `,`.
- `slice(start, end)` (الـ `end` لا تشمل الـ) قص جزء من المصفوفة في مصفوفة جديدة.

```
var myArr = ["apple", "banana", "mango"];

// concat
var newArray = myArr.concat(fruitsObj);
console.log(newArray); // ['apple', 'banana', 'mango', 'mango', 'watermelon']
console.log(myArr); // المصفوفة الأصلية لم تتأثر

// join
```

```

console.log(myArr.join("-")); // "apple-banana-mango" (String)
var newArrStr = myArr.join("-");
console.log(newArrStr.split("-")); // ['apple', 'banana', 'mango'] (رجعناها مصفوفة تاني)

// toString
console.log(myArr.toString()); // "apple,banana,mango"

// slice
var slicedArray = myArr.slice(0, 2);
console.log(slicedArray); // ['apple', 'banana']

```

## 2. ميثودز تُعدل في المصفوفة الأصلية فوراً (Mutable - Modifies Original Array)

- `pop()`: تحذف آخر عنصر وترجعه.
- `push(item)`: تضيف عنصر في آخر المصفوفة.
- `shift()`: تحذف أول عنصر وترجعه.
- `unshift(item)`: تضيف عنصر في أول المصفوفة.
- `reverse()`: تشقلب ترتيب العناصر.
- `sort()`: الترتيب الأبجدي/الرقمي.
- `splice(start, deleteCount, items_to_add)`: إضافة أو حذف عناصر من أي مكان.

### JavaScript

```

// pop
var lastElement = myArr.pop();
console.log(lastElement); // "mango"
console.log(myArr); // ['apple', 'banana'] (اتعدلت)

// push
myArr.push("watermelon");
console.log(myArr); // ['apple', 'banana', 'watermelon']

// shift
var firstElement = myArr.shift();
console.log(firstElement); // "apple"
console.log(myArr); // ['banana', 'watermelon']

// unshift
myArr.unshift("strawberry");
console.log(myArr); // ['strawberry', 'banana', 'watermelon']

// reverse

```

```
myArr.reverse();
console.log(myArr); // ['watermelon', 'banana', 'strawberry']
```

### ⚠ تنبيه خطير جداً بخصوص `sort()` بالأرقام

دالة `sort()` الافتراضية بترتب بناءً على الأبجدية (**String Unicode**)، عشان كده الأرقام مش بتترتب صح (مثلاً 10 هتيجي قبل 3 لأنها بتبدأ بـ 1!).

**الحل:** لازم نباضي ليها (Callback Function) المقارنة الذكية: `a - b <= (a, b)`.

## JavaScript

```
var numbers = [5, 9, 3, 10, 6, 7, 4, 1];
numbers.sort();
console.log(numbers); // الناتج: [1, 10, 3, 4, 5, 6, 7, 9] (ترتيب خاطئ ⚠ رياضيًا)

// الترتيب الصحيح للأرقام:
numbers.sort((a, b) => a - b);
console.log(numbers); // الناتج: [1, 3, 4, 5, 6, 7, 9, 10]

// قاعدة المقارنة (a - b):
// b قبل a إذا كان الناتج > 0 -> يضع
الناتج اقل من الصفر تصاعدي اكبر من الصفر تنازلي
// a قبل b إذا كان الناتج < 0 -> يضع
// إذا كان الناتج = 0 -> يتركهم كما هم
```

- **splice (الجوكر):**

## JavaScript

```
// الاندكس اللي هبدأ عنده, عدد العناصر اللي هتمسحها, الحاجات اللي
(هتضيفها)
myArr.splice(1, 0, "mango", "mango"); // يمسح 0 عنصر، ويضيف
mango كلمتين
console.log(myArr);
```

## 6. المسألة البرمجية المقترحة في السكشن (تلميح للتاسك)

في آخر السكشن مكتوب مصفوفة أرقام كومت كذا:

## JavaScript



```
var myArray = [1, 2, 3, 17, 30, 50, 80, 10, 20, 5, 6, 7];
// أو حاصل الضرب (Sum) المطلوب عادة في الأفكار دي: حساب المجموع
// أو القسمة (Multiplication).
// فكرة الحل دائماً بتبدأ بفرض مخزن ابتدائي
// (initial sum = 0) للجمع الإجمالي: المجموع الابتدائي = 0
// عشان محايد ضربى (initial multi = 1) للضرب الإجمالي: الضرب الابتدائي = 1.
```